

Lahore Garrison University

Compiler Construction

Semester Project

Mini Python Compiler

Submitted to:

Miss Saba Mohsin

Submitted by:

Asma Qamar

ROLL NO. Fa-2023/BSCS/079

Saheefa khan

ROLL NO. Fa-2023/BSCS/643

Saliha khan

ROLL NO. Fa-2023/BSCS/046

Bisma Bakhtawar

ROLL NO. Fa-2023/BSCS/612

Class: BSCS-6th Semester

Section: B

1. Abstract

This report presents the design and implementation of a Mini Python Compiler created for the Compiler Construction term project. The compiler works on a selected subset of the Python language and demonstrates the major theoretical and practical phases of a compiler. It begins with lexical analysis, where the source code is converted into tokens and lexemes. It then performs syntax analysis using Python abstract syntax structures, displays a readable parse tree, performs semantic checking, builds a symbol table, generates intermediate code in the form of Three Address Code (TAC), applies optimization techniques, and provides meaningful error handling with line numbers and suggestions. The purpose of this project is not to replace a production Python interpreter, but to demonstrate how compiler phases interact and how source code is transformed from high-level statements into structured intermediate representation.

2. Introduction

A compiler is a system program that translates source code written in a high-level programming language into another representation, such as machine code, bytecode, or intermediate code. In compiler construction, the source program is usually processed through multiple phases. Each phase takes input from the previous phase, performs a specific transformation or verification, and passes output to the next phase.

The Mini Python Compiler developed in this project is designed as an educational compiler. It accepts Python-like source programs containing assignments, arithmetic expressions, comparison expressions, if statements, and print statements. It then prints a phase-wise report directly in the command prompt.

3. Project Objectives

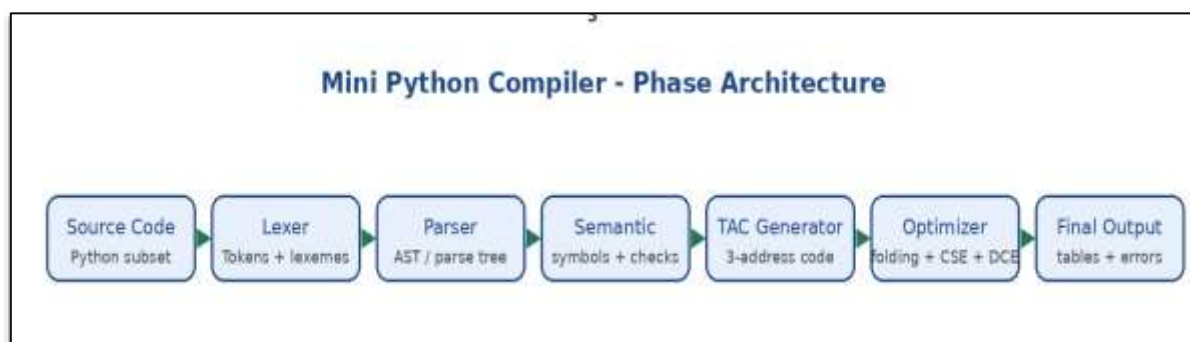
- To implement a compiler-like system using Python 3.
- To accept a meaningful subset of Python as the source language.
- To show lexical analysis in the form of token and lexeme tables.
- To display syntax analysis and a readable parse tree.
- To perform semantic checks such as undeclared variable detection and type mismatch detection.
- To create a well-formatted symbol table with scope, variable name, type, value, offset, width, and line number.
- To generate Three Address Code as intermediate representation.
- To apply optimization techniques such as constant folding, common subexpression elimination, and dead code elimination.
- To handle errors with proper line numbers and suggestions.

4. Tools and Technologies Used

Tool / Technology	Purpose	Reason for Selection
Python 3	Compiler implementation language	Simple syntax, strong standard library, easy file handling
tokenize module	Lexical analysis	Provides Python tokens, lexemes, line numbers, and columns
ast module	Syntax analysis and parse tree	Generates Python abstract syntax tree for valid source code
Command Prompt	Running compiler demo	Easy to demonstrate using two commands
Notepad / VS Code	Editing source files	Easy source code editing

5. System Architecture

The compiler follows a classical compiler pipeline. The source program is first passed to the lexer. The lexer creates tokens. These tokens and the original source are then parsed. After syntax validation, semantic analysis checks variable declarations, variable usage, and expression types. The symbol table stores information about identifiers. TAC generation converts complex expressions into simple three-address instructions. Finally, optimization improves the generated intermediate code.



Phase	Input	Output
Lexical Analysis	Python source code	Token table and token summary
Syntax Analysis	Source code tokens/structure	Parse success and syntax tree
Semantic Analysis	AST and symbol usage	Semantic success or error list
Symbol Table	Identifiers and expressions	Formatted variable table
TAC Generation	AST expressions and statements	Three Address Code
Optimization	Raw TAC	Optimized TAC and report
Error Handling	Exceptions and validation results	Line-wise error messages and suggestions

11. Limitations and Future Work

The Mini Python Compiler is designed for educational demonstration, so it intentionally supports a limited subset of Python. However, it successfully demonstrates all major compiler construction phases required for a term project.

- The compiler supports a subset of Python.
- It generates intermediate code but does not generate executable machine code.
- The type system is basic and mainly focused on numbers and strings.
- Scope handling is simplified for demonstration purposes.
- The optimizer demonstrates core ideas but is not as advanced as production compiler optimizers.

Future Enhancements

- Add support for while loops and for loops.
- Add complete function definition and function call checking.
- Add a graphical user interface to enter source code and view output.
- Generate MIPS, assembly, or bytecode from TAC.
- Add live highlighting for syntax and semantic errors.
- Export compiler output directly into PDF or HTML report format.

12. Conclusion

This project successfully demonstrates the working of a compiler using Python. The Mini Python Compiler accepts Python source code, performs lexical analysis, syntax analysis, semantic checking, symbol table generation, TAC generation, optimization, and error handling. The output is well formatted and suitable for a classroom demonstration. The project is valuable because it connects theoretical compiler construction concepts with practical implementation.

The strongest part of the project is that every phase is visible. A teacher or evaluator can see tokens, parse tree, semantic status, symbol table, TAC, optimized TAC, and error messages. This makes the project complete, understandable, and presentation-ready.

Lexical analysis

1. LEXICAL ANALYSIS					
No.	Lexeme	Token	Type	Line	Col
1	a	IDENTIFIER	Identifier	1	1
2	=	OPERATOR	Operator	1	3
3	5	NUMBER	Constant	1	5
4	\n	NEWLINE	Line Break	1	6
5	b	IDENTIFIER	Identifier	2	1
6	=	OPERATOR	Operator	2	3
7	10	NUMBER	Constant	2	5
8	\n	NEWLINE	Line Break	2	7
9	c	IDENTIFIER	Identifier	3	1
10	=	OPERATOR	Operator	3	3
11	20	NUMBER	Constant	3	5
12	\n	NEWLINE	Line Break	3	7
13	d	IDENTIFIER	Identifier	4	1
14	=	OPERATOR	Operator	4	3
15	4	NUMBER	Constant	4	5
16	\n	NEWLINE	Line Break	4	6
17	x	IDENTIFIER	Identifier	6	1
18	=	OPERATOR	Operator	6	3
19	(	OPERATOR	Operator	6	5
20	a	IDENTIFIER	Identifier	6	6
21	+	OPERATOR	Operator	6	8
22	b	IDENTIFIER	Identifier	6	10
23	)	OPERATOR	Operator	6	11
24	+	OPERATOR	Operator	6	13
25	(	OPERATOR	Operator	6	15
26	c	IDENTIFIER	Identifier	6	16
27	-	OPERATOR	Operator	6	18
28	d	IDENTIFIER	Identifier	6	20
29	)	OPERATOR	Operator	6	21
30	\n	NEWLINE	Line Break	6	22
31	y	IDENTIFIER	Identifier	7	1
32	=	OPERATOR	Operator	7	3

47	3	NUMBER	Constant	9	14
48	\n	NEWLINE	Line Break	9	15
49	result	IDENTIFIER	Identifier	10	1
50	=	OPERATOR	Operator	10	8
51	x	IDENTIFIER	Identifier	10	10
52	+	OPERATOR	Operator	10	12
53	folded	IDENTIFIER	Identifier	10	14
54	\n	NEWLINE	Line Break	10	20
55	unused	IDENTIFIER	Identifier	11	1
56	=	OPERATOR	Operator	11	8
57	100	NUMBER	Constant	11	10
58	\n	NEWLINE	Line Break	11	13
59	if	KEYWORD	Keyword	13	1
60	result	IDENTIFIER	Identifier	13	4
61	>	OPERATOR	Operator	13	11
62	50	NUMBER	Constant	13	13
63	:	OPERATOR	Operator	13	15
64	\n	NEWLINE	Line Break	13	16
65	INDENT	INDENT	Indentation	14	1
66	message	IDENTIFIER	Identifier	14	5
67	=	OPERATOR	Operator	14	13
68	result	IDENTIFIER	Identifier	14	15
69	+	OPERATOR	Operator	14	22
70	1	NUMBER	Constant	14	24
71	\n	NEWLINE	Line Break	14	25
72	print	BUILTIN	Built-in Function	15	5
73	(	OPERATOR	Operator	15	10
74	message	IDENTIFIER	Identifier	15	11
75	)	OPERATOR	Operator	15	18
76	\n	NEWLINE	Line Break	15	19
77	DEDENT	DEDENT	Indentation	16	1

Token Summary

Token Summary	
Category	Values
Keywords	if
Identifiers	a, b, c, d, folded, message, result, unused, x, y, z
Operators	(,), *, +, -, :, =, >
Constants	1, 10, 100, 20, 3, 4, 5, 50

Syntax analysis

```
2. SYNTAX ANALYSIS
=====
Parse Status: SUCCESS
Parser Message: Source code has valid Python syntax.

Parse Tree / Syntax Tree
-----
Program
├── Assignment Statement: a
│   ├── Identifier: a
│   └── Constant: 5
├── Assignment Statement: b
│   ├── Identifier: b
│   └── Constant: 10
├── Assignment Statement: c
│   ├── Identifier: c
│   └── Constant: 20
├── Assignment Statement: d
│   ├── Identifier: d
│   └── Constant: 4
├── Assignment Statement: x
│   ├── Identifier: x
│   └── Binary Expression: +
│       ├── Binary Expression: +
│       │   ├── Identifier: a
│       │   └── Identifier: b
│       └── Binary Expression: -
│           ├── Identifier: c
│           └── Identifier: d
├── Assignment Statement: y
│   ├── Identifier: y
│   └── Binary Expression: +
│       ├── Identifier: a
│       └── Identifier: b
└── Assignment Statement: z
    ├── Identifier: z
    └── Binary Expression: +
        └── Identifier: a
```

```

└─ Constant: 5
└─ Assignment Statement: b
  └─ Identifier: b
  └─ Constant: 10
└─ Assignment Statement: c
  └─ Identifier: c
  └─ Constant: 20
└─ Assignment Statement: d
  └─ Identifier: d
  └─ Constant: 4
└─ Assignment Statement: x
  └─ Identifier: x
  └─ Binary Expression: *
    └─ Binary Expression: +
      └─ Identifier: a
      └─ Identifier: b
    └─ Binary Expression: -
      └─ Identifier: c
      └─ Identifier: d
└─ Assignment Statement: y
  └─ Identifier: y
  └─ Binary Expression: +
    └─ Identifier: a
    └─ Identifier: b
└─ Assignment Statement: z
  └─ Identifier: z
  └─ Binary Expression: +
    └─ Identifier: a
    └─ Identifier: b
└─ Assignment Statement: folded
  └─ Identifier: folded
  └─ Binary Expression: +
    └─ Constant: 5
    └─ Constant: 3

```

```

└─ Assignment Statement: d
  └─ Identifier: d
  └─ Constant: 4
└─ Assignment Statement: x
  └─ Identifier: x
  └─ Binary Expression: *
    └─ Binary Expression: +
      └─ Identifier: a
      └─ Identifier: b
    └─ Binary Expression: -
      └─ Identifier: c
      └─ Identifier: d
└─ Assignment Statement: y
  └─ Identifier: y
  └─ Binary Expression: +
    └─ Identifier: a
    └─ Identifier: b
└─ Assignment Statement: z
  └─ Identifier: z
  └─ Binary Expression: +
    └─ Identifier: a
    └─ Identifier: b
└─ Assignment Statement: folded
  └─ Identifier: folded
  └─ Binary Expression: +
    └─ Constant: 5
    └─ Constant: 3
└─ Assignment Statement: result
  └─ Identifier: result
  └─ Binary Expression: +
    └─ Identifier: x
    └─ Identifier: folded

```

Semantic Analysis

3. SEMANTIC ANALYSIS

Semantic Status: SUCCESS

No undeclared variables or type mismatch errors found.

Symbol Table

4. SYMBOL TABLE

Scope	Variable	Type	Value	Offset	Width	Line	Details
global	a	int	5	0	4	1	Declared and initialized
global	b	int	10	4	4	2	Declared and initialized
global	c	int	20	8	4	3	Declared and initialized
global	d	int	4	12	4	4	Declared and initialized
global	x	int	240	16	4	6	Declared and initialized
global	y	int	15	20	4	7	Declared and initialized
global	z	int	15	24	4	8	Declared and initialized
global	folded	int	8	28	4	9	Declared and initialized
global	result	int	248	32	4	10	Declared and initialized
global	unused	int	100	36	4	11	Declared and initialized
global	message	int	249	40	4	14	Declared and initialized

Intermediate code generation

5. INTERMEDIATE CODE GENERATION (TAC)

Line	Result	Arg1	Operator	Arg2	Three Address Code
1	a	5	=		a = 5
2	b	10	=		b = 10
3	c	20	=		c = 20
4	d	4	=		d = 4
5	t1	a	+	b	t1 = a + b
6	t2	c	-	d	t2 = c - d
7	t3	t1	*	t2	t3 = t1 * t2
8	x	t3	=		x = t3
9	t4	a	+	b	t4 = a + b
10	y	t4	=		y = t4
11	t5	a	+	b	t5 = a + b
12	z	t5	=		z = t5
13	t6	5	+	3	t6 = 5 + 3
14	folded	t6	=		folded = t6
15	t7	x	+	folded	t7 = x + folded
16	result	t7	=		result = t7
17	unused	100	=		unused = 100
18	t8	result	>	50	t8 = result > 50
19		t8	ifFalse	L1	ifFalse t8 goto L1
20	t9	result	+	1	t9 = result + 1
21	message	t9	=		message = t9
22		message	param		param message
23		print	call	1	call print, 1
24	L1		label		L1:

Optimization phase

6. OPTIMIZATION PHASE		
Optimization Report		
Technique	Before	After
Common Subexpression	t4 = a + b	t4 = t1
Common Subexpression	t5 = a + b	t5 = t1
Constant Folding	t6 = 5 + 3	t6 = 8
Dead Code Elimination	y = t1	Removed unused assignment
Dead Code Elimination	z = t1	Removed unused assignment
Dead Code Elimination	unused = 100	Removed unused assignment

Optimized Tac

Optimized TAC					
Line	Result	Arg1	Operator	Arg2	Optimized Code
1	a	5	=		a = 5
2	b	10	=		b = 10
3	c	20	=		c = 20
4	d	4	=		d = 4
5	t1	a	+	b	t1 = a + b
6	t2	c	-	d	t2 = c - d
7	t3	t1	*	t2	t3 = t1 * t2
8	x	t3	=		x = t3
9	t4	t1	=		t4 = t1
10	t5	t1	=		t5 = t1
11	t6	8	=		t6 = 8
12	folded	8	=		folded = 8
13	t7	x	+	folded	t7 = x + folded
14	result	t7	=		result = t7
15	t8	result	>	50	t8 = result > 50
16		t8	iffalse	L1	iffalse t8 goto L1
17	t9	result	+	1	t9 = result + 1
18	message	t9	=		message = t9
19		message	param		param message
20		print	call	1	call print, 1
21	L1		label		L1:

Error handling

7. ERROR HANDLING	
No syntax or semantic errors found.	
Compilation completed successfully.	